

Complete Kinematic Model

4-Wheel Synchronized Steering Robot for Maze Navigation

Kinematic Analysis and Implementation Guide

November 1, 2025

Contents

1	Robot Configuration Overview	3
1.1	Physical Description	3
1.2	Physical Dimensions	3
1.3	Robot Coordinate Frame	3
2	Kinematic Constraints	3
2.1	Mechanical Linkage	3
2.2	Wheel Positions	4
3	Individual Wheel Kinematics	4
3.1	Wheel Velocity Constraint	4
3.2	No-Slip Constraint	5
4	Operating Modes	5
4.1	Mode 1: Pivot Mode	5
4.1.1	Geometry	5
4.1.2	Kinematic Equations	5
4.1.3	Implementation	6
4.2	Mode 2: Crab Mode	6
4.2.1	Geometry	6
4.2.2	Kinematic Equations	7
4.2.3	Special Cases	7
4.2.4	Implementation	7
4.3	Mode 3: Ackermann Mode (Approximate)	7
4.3.1	Challenge with Synchronized Steering	8
4.3.2	Approximate Ackermann	8
4.3.3	Wheel Speed Differential	8
4.3.4	Practical Use in Maze	8
5	Forward Kinematics	8
5.1	General Form	8
5.2	Discrete Time Update	9
5.3	Mode-Specific Forward Kinematics	9
5.3.1	Pivot Mode	9
5.3.2	Crab Mode	9
5.3.3	Ackermann Mode	9

6	Inverse Kinematics	9
6.1	Problem Statement	9
6.2	Steering Angle Computation	10
6.2.1	Pure Rotation ($v_x = v_y = 0, \omega \neq 0$)	10
6.2.2	Pure Translation ($\omega = 0, v_x$ or $v_y \neq 0$)	10
6.2.3	Combined Motion ($v \neq 0, \omega \neq 0$)	10
6.3	Wheel Speed Computation	10
6.3.1	Mode-Specific Formulas	10
7	IMU Integration for Heading Correction	10
7.1	Sensor Fusion Strategy	10
7.2	Complementary Filter	11
7.3	Integration with Odometry	11
7.4	Kalman Filter (Advanced)	11
8	Odometry from Encoders	11
8.1	Encoder Measurements	11
8.2	Dead Reckoning	11
8.3	Odometry Equations	12
9	Control Implementation	12
9.1	System Architecture	12
9.2	Control Loop Structure	12
9.3	Arduino Nano 33 BLE Feasibility	13
10	Maze Navigation Strategy	13
10.1	Navigation Phases	13
10.1.1	Phase 1: Straight Corridor	13
10.1.2	Phase 2: 90° Turn	14
10.1.3	Phase 3: Centering in Corridor	14
10.2	Tight Gap Navigation	14
11	Implementation Checklist	14
11.1	Hardware Setup	14
11.2	Software Development	15
11.3	Testing Phases	15
12	Summary	15
12.1	Key Equations	15
12.2	Recommended Parameter Values	15
13	Conclusion	16

1 Robot Configuration Overview

1.1 Physical Description

This document presents the complete kinematic model for a 4-wheel robot with synchronized steering designed for precise maze navigation. The robot features:

- 4 independently driven wheels at the corners of a rectangular base
- Synchronized steering via mechanical gear linkage
- 9-DOF IMU for heading correction
- Encoders on all motors
- Target gaps: 180mm (with 119mm robot footprint)

1.2 Physical Dimensions

Parameter	Symbol	Value
Base Width	w	0.119 m
Base Length	l	0.119 m
Track Width	t	0.074 m
Wheel Radius	r_{wheel}	0.0127 m (12.7 mm)
Steering Range	θ_s	-90 to $+90$

Table 1: Robot Physical Parameters

1.3 Robot Coordinate Frame

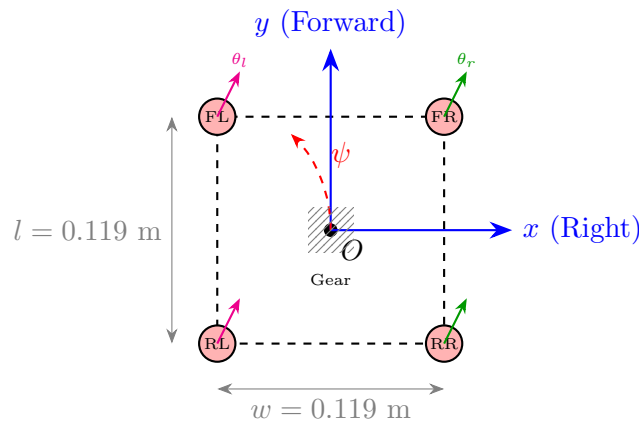


Figure 1: Robot coordinate frame and wheel configuration. Magenta arrows show left wheel steering (θ_l), green arrows show right wheel steering (θ_r). The middle gear synchronizes: $\theta_l = -\theta_r$.

2 Kinematic Constraints

2.1 Mechanical Linkage

The key constraint of this robot is the mechanical synchronization via a middle gear:

$$\boxed{\theta_l = -\theta_r} \quad (1)$$

We can define a single steering control variable θ_s :

$$\text{Left wheels (FL, RL): } \theta_l = +\theta_s \quad (2)$$

$$\text{Right wheels (FR, RR): } \theta_r = -\theta_s \quad (3)$$

This constraint reduces the system to **one steering degree of freedom**, enabling three distinct operating modes:

- $\theta_s > 0$: Counterclockwise (CCW) pivot
- $\theta_s = 0$: Parallel wheels (crab/straight motion)
- $\theta_s < 0$: Clockwise (CW) pivot

2.2 Wheel Positions

Relative to the robot center, the four wheel positions are:

$$\text{FL: } \mathbf{p}_{FL} = \begin{bmatrix} -w/2 \\ +l/2 \end{bmatrix} = \begin{bmatrix} -0.0595 \\ +0.0595 \end{bmatrix} \text{ m} \quad (4)$$

$$\text{FR: } \mathbf{p}_{FR} = \begin{bmatrix} +w/2 \\ +l/2 \end{bmatrix} = \begin{bmatrix} +0.0595 \\ +0.0595 \end{bmatrix} \text{ m} \quad (5)$$

$$\text{RL: } \mathbf{p}_{RL} = \begin{bmatrix} -w/2 \\ -l/2 \end{bmatrix} = \begin{bmatrix} -0.0595 \\ -0.0595 \end{bmatrix} \text{ m} \quad (6)$$

$$\text{RR: } \mathbf{p}_{RR} = \begin{bmatrix} +w/2 \\ -l/2 \end{bmatrix} = \begin{bmatrix} +0.0595 \\ -0.0595 \end{bmatrix} \text{ m} \quad (7)$$

3 Individual Wheel Kinematics

3.1 Wheel Velocity Constraint

For each wheel i at position (x_i, y_i) with steering angle θ_i , the wheel must roll in the direction of its orientation without slipping.

The wheel direction unit vector is:

$$\hat{\mathbf{u}}_i = \begin{bmatrix} \cos(\theta_i) \\ \sin(\theta_i) \end{bmatrix} \quad (8)$$

The velocity of the wheel contact point due to robot motion is:

$$\mathbf{v}_i = \begin{bmatrix} v_x \\ v_y \end{bmatrix} + \omega \times \begin{bmatrix} x_i \\ y_i \end{bmatrix} = \begin{bmatrix} v_x - \omega y_i \\ v_y + \omega x_i \end{bmatrix} \quad (9)$$

where:

- v_x : lateral velocity (robot frame)
- v_y : forward velocity (robot frame)
- ω : angular velocity

3.2 No-Slip Constraint

The wheel velocity must be parallel to the wheel direction (no lateral slip):

$$\mathbf{v}_i \parallel \hat{\mathbf{u}}_i \quad (10)$$

This can be expressed as:

$$\mathbf{v}_i = \lambda_i \hat{\mathbf{u}}_i \quad (11)$$

where λ_i is the wheel rolling speed (scalar).

Alternatively, the perpendicular component must be zero:

$$\hat{\mathbf{u}}_i^\perp \cdot \mathbf{v}_i = 0 \quad (12)$$

where $\hat{\mathbf{u}}_i^\perp = \begin{bmatrix} -\sin(\theta_i) \\ \cos(\theta_i) \end{bmatrix}$ is the perpendicular direction.

3.3 Wheel Angular to Linear Velocity Conversion

Each wheel has a radius $r_{wheel} = 0.0127$ m (12.7 mm). The relationship between wheel angular velocity ω_{wheel} (in rad/s) and linear velocity v_{wheel} (in m/s) is:

$$v_{wheel} = \omega_{wheel} \cdot r_{wheel} \quad (13)$$

Conversely:

$$\omega_{wheel} = \frac{v_{wheel}}{r_{wheel}} \quad (14)$$

For encoder-based motor control, if the encoder provides counts per revolution CPR :

$$v_{wheel} = \frac{\Delta counts}{CPR} \cdot 2\pi r_{wheel} \cdot \frac{1}{\Delta t} \quad (15)$$

Example: For a wheel speed of $v_{wheel} = 0.05$ m/s:

$$\omega_{wheel} = \frac{0.05}{0.0127} = 3.94 \text{ rad/s} = 37.6 \text{ RPM} \quad (16)$$

4 Operating Modes

4.1 Mode 1: Pivot Mode

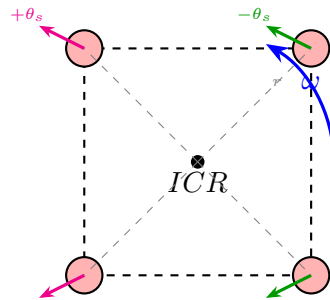


Figure 2: Pivot Mode: All wheels point toward/away from the center ICR. Robot rotates in place. $\theta_l = +\theta_s$, $\theta_r = -\theta_s$.

4.1.1 Geometry

In pivot mode, the Instantaneous Center of Rotation (ICR) is at the robot center. Each wheel must point tangent to a circle centered at the origin.

For the left wheels (FL, RL) at positions $(-w/2, \pm l/2)$:

$$\tan(\theta_l) = \frac{x_i}{-y_i} \quad (17)$$

For FL wheel:

$$\theta_{FL} = \arctan\left(\frac{-w/2}{-l/2}\right) = \arctan\left(\frac{w}{l}\right) \quad (18)$$

Since $w = l = 0.119$ m:

$$\boxed{\theta_s = \arctan(1) = 45} \quad (19)$$

4.1.2 Kinematic Equations

With $\theta_s = 45$, the robot velocity is:

$$v_x = 0 \quad (20)$$

$$v_y = 0 \quad (21)$$

$$\omega = \text{angular velocity (controllable)} \quad (22)$$

The wheel rolling speeds are:

$$v_{wheel,i} = \omega \cdot r_i \quad (23)$$

where r_i is the distance from wheel to ICR:

$$r = \sqrt{\left(\frac{w}{2}\right)^2 + \left(\frac{l}{2}\right)^2} = \frac{\sqrt{w^2 + l^2}}{2} = \frac{0.119\sqrt{2}}{2} = 0.0842 \text{ m} \quad (24)$$

4.1.3 Implementation

To rotate at angular velocity ω :

1. Set steering: $\theta_s = 45$
2. Set wheel speeds:

$$v_{FL} = +\omega \cdot r \quad (25)$$

$$v_{FR} = -\omega \cdot r \quad (26)$$

$$v_{RL} = +\omega \cdot r \quad (27)$$

$$v_{RR} = -\omega \cdot r \quad (28)$$

For a 90° turn at moderate speed (0.1 rad/s):

- Time required: $t = \frac{\pi/2}{0.1} = 15.7$ seconds
- Wheel speeds: $v = 0.1 \times 0.0842 = 0.00842$ m/s = 8.42 mm/s
- Wheel angular velocity: $\omega_{wheel} = \frac{0.00842}{0.0127} = 0.663$ rad/s = 6.33 RPM

4.2 Mode 2: Crab Mode

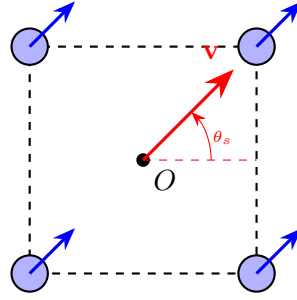


Figure 3: Crab Mode: All wheels parallel, robot translates at angle θ_s without rotation. $\theta_l = \theta_r = \theta_s$.

4.2.1 Geometry

In crab mode, all wheels are parallel:

$$\theta_l = \theta_r = \theta_s \quad (29)$$

The robot translates without rotation ($\omega = 0$) at an angle θ_s relative to the robot's forward axis.

4.2.2 Kinematic Equations

The robot velocity in the robot frame:

$$v_x = v \cos(\theta_s) \quad (30)$$

$$v_y = v \sin(\theta_s) \quad (31)$$

$$\omega = 0 \quad (32)$$

where v is the desired speed magnitude.

All wheels have the same rolling speed:

$$v_{FL} = v_{FR} = v_{RL} = v_{RR} = v \quad (33)$$

4.2.3 Special Cases

- $\theta_s = 0$: Pure forward motion ($v_x = v, v_y = 0$)
- $\theta_s = 90$: Pure lateral motion ($v_x = 0, v_y = v$)
- $\theta_s = 45$: Diagonal motion ($v_x = v_y = v/\sqrt{2}$)

4.2.4 Implementation

For straight-line motion in direction θ_s at speed v :

1. Set steering: θ_s (all wheels)
2. Set all wheel speeds: $v_{wheel} = v$
3. Monitor distance: $d = \int v dt$

Maze Navigation Example: To move forward 0.3 m at 0.05 m/s:

- Steering: $\theta_s = 0$
- Wheel speeds: $v = 0.05$ m/s
- Wheel angular velocity: $\omega_{wheel} = \frac{0.05}{0.0127} = 3.94$ rad/s = 37.6 RPM
- Time: $t = 0.3/0.05 = 6$ seconds

4.3 Mode 3: Ackermann Mode (Approximate)

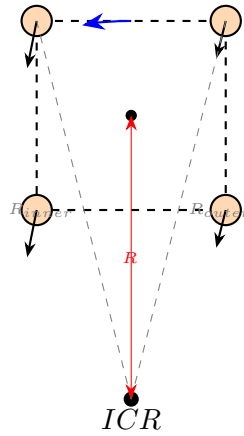


Figure 4: Ackermann Mode: ICR is to the side, wheels steer to point toward ICR. Left/right wheels have different steering angles due to mechanical constraint. This is an approximation of true Ackermann geometry.

4.3.1 Challenge with Synchronized Steering

True Ackermann steering requires:

$$\cot(\theta_{inner}) - \cot(\theta_{outer}) = \frac{w}{R} \quad (34)$$

However, our mechanical constraint forces: $\theta_l = -\theta_r$

This means we **cannot achieve perfect Ackermann geometry**. Instead, we approximate it by choosing a compromise steering angle.

4.3.2 Approximate Ackermann

For a desired turning radius R , we use:

$$\theta_s \approx \arctan\left(\frac{L}{R}\right) \quad (35)$$

where $L = \sqrt{w^2 + l^2}/2 = 0.0842$ m is the effective wheelbase.

The angular velocity is:

$$\omega = \frac{v}{R} \quad (36)$$

4.3.3 Wheel Speed Differential

To maintain proper rolling without slip, wheels on the inside of the turn must move slower:

$$v_{inner} = v \left(1 - \frac{w}{2R}\right) \quad (37)$$

$$v_{outer} = v \left(1 + \frac{w}{2R}\right) \quad (38)$$

For left wheels (assuming CCW turn):

$$v_{FL} = v_{RL} = \omega(R - w/2) \quad (39)$$

$$v_{FR} = v_{RR} = \omega(R + w/2) \quad (40)$$

4.3.4 Practical Use in Maze

In tight maze spaces (180mm gaps), Ackermann mode has limited utility. It's more useful for:

- Smooth transitions between corridors
- Energy-efficient turns in open areas
- Reduced wheel slip compared to pivot mode

For sharp 90° turns in narrow corridors, **pivot mode is preferred**.

5 Forward Kinematics

5.1 General Form

Given the robot's velocity in the robot frame $\mathbf{V}_R = [v_x, v_y, \omega]^T$, we can compute the position change:

Robot frame velocities:

$$\begin{bmatrix} \dot{x}_R \\ \dot{y}_R \\ \dot{\psi} \end{bmatrix} = \begin{bmatrix} v_x \\ v_y \\ \omega \end{bmatrix} \quad (41)$$

World frame velocities:

$$\begin{bmatrix} \dot{x}_W \\ \dot{y}_W \\ \dot{\psi} \end{bmatrix} = \begin{bmatrix} \cos \psi & -\sin \psi & 0 \\ \sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} v_x \\ v_y \\ \omega \end{bmatrix} \quad (42)$$

5.2 Discrete Time Update

For a control period Δt :

$$x(t + \Delta t) = x(t) + [v_x \cos \psi - v_y \sin \psi] \Delta t \quad (43)$$

$$y(t + \Delta t) = y(t) + [v_x \sin \psi + v_y \cos \psi] \Delta t \quad (44)$$

$$\psi(t + \Delta t) = \psi(t) + \omega \Delta t \quad (45)$$

5.3 Mode-Specific Forward Kinematics

5.3.1 Pivot Mode

$$v_x = 0 \quad (46)$$

$$v_y = 0 \quad (47)$$

$$\omega = \text{measured from encoders or IMU} \quad (48)$$

Result: Pure rotation, no translation.

5.3.2 Crab Mode

$$v_x = v \cos \theta_s \quad (49)$$

$$v_y = v \sin \theta_s \quad (50)$$

$$\omega = 0 \quad (51)$$

Result: Pure translation at angle θ_s .

5.3.3 Ackermann Mode

$$v_x = -v \sin \theta_s \cdot \frac{w}{2R} \quad (52)$$

$$v_y = v \cos \theta_s \quad (53)$$

$$\omega = \frac{v}{R} \quad (54)$$

Result: Combined translation and rotation.

6 Inverse Kinematics

6.1 Problem Statement

Given desired robot velocity $\mathbf{V}_R = [v_x, v_y, \omega]^T$, compute:

1. Steering angles: θ_s
2. Wheel speeds: $v_{FL}, v_{FR}, v_{RL}, v_{RR}$

6.2 Steering Angle Computation

6.2.1 Pure Rotation ($v_x = v_y = 0, \omega \neq 0$)

$$\boxed{\theta_s = 45 \cdot \text{sign}(\omega)} \quad (55)$$

6.2.2 Pure Translation ($\omega = 0, v_x$ or $v_y \neq 0$)

$$\boxed{\theta_s = \arctan 2(v_y, v_x)} \quad (56)$$

6.2.3 Combined Motion ($v \neq 0, \omega \neq 0$)

Use Ackermann approximation:

$$\theta_s = \arctan \left(\frac{\omega \cdot L}{|v|} \right) \cdot \text{sign}(v) \quad (57)$$

where $v = \sqrt{v_x^2 + v_y^2}$

6.3 Wheel Speed Computation

For each wheel at position (x_i, y_i) with steering angle θ_i :

$$v_i = \frac{(v_x - \omega y_i) \cos \theta_i + (v_y + \omega x_i) \sin \theta_i}{1} \quad (58)$$

6.3.1 Mode-Specific Formulas

Pivot Mode:

$$v_i = \omega \sqrt{x_i^2 + y_i^2} \cdot \text{sign}_i \quad (59)$$

where sign_i depends on wheel position relative to rotation direction.

Crab Mode:

$$v_{FL} = v_{FR} = v_{RL} = v_{RR} = v \quad (60)$$

Ackermann Mode:

$$v_{FL} = v_{RL} = \omega(R - w/2) \quad (61)$$

$$v_{FR} = v_{RR} = \omega(R + w/2) \quad (62)$$

7 IMU Integration for Heading Correction

7.1 Sensor Fusion Strategy

The 9-DOF IMU provides:

- Gyroscope: Angular velocity ω_{gyro} (drift over time)
- Magnetometer: Absolute heading ψ_{mag} (noisy, affected by metal)
- Accelerometer: For tilt compensation

7.2 Complementary Filter

Combine gyro (high-frequency accurate) with magnetometer (low-frequency reference):

$$\boxed{\psi(t + \Delta t) = \alpha[\psi(t) + \omega_{gyro}\Delta t] + (1 - \alpha)\psi_{mag}} \quad (63)$$

Recommended: $\alpha = 0.98$ at 20 Hz update rate.

7.3 Integration with Odometry

1. **100 Hz loop:** Update position using encoder-based odometry
2. **20 Hz loop:** Correct heading using IMU
3. **Correction:**

$$\Delta\psi_{correction} = \psi_{IMU} - \psi_{odometry} \quad (64)$$

Apply small correction to odometry heading to prevent drift

7.4 Kalman Filter (Advanced)

For better performance, implement an Extended Kalman Filter (EKF):

State vector:

$$\mathbf{x} = [x, y, \psi, v_x, v_y, \omega, b_\omega]^T \quad (65)$$

where b_ω is gyro bias.

Prediction (from encoders):

$$\hat{\mathbf{x}}_{k|k-1} = f(\mathbf{x}_{k-1}, \mathbf{u}_k) \quad (66)$$

$$\mathbf{P}_{k|k-1} = \mathbf{F}_k \mathbf{P}_{k-1} \mathbf{F}_k^T + \mathbf{Q}_k \quad (67)$$

Update (from IMU):

$$\mathbf{K}_k = \mathbf{P}_{k|k-1} \mathbf{H}_k^T (\mathbf{H}_k \mathbf{P}_{k|k-1} \mathbf{H}_k^T + \mathbf{R}_k)^{-1} \quad (68)$$

$$\hat{\mathbf{x}}_k = \hat{\mathbf{x}}_{k|k-1} + \mathbf{K}_k (\mathbf{z}_k - h(\hat{\mathbf{x}}_{k|k-1})) \quad (69)$$

$$\mathbf{P}_k = (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k) \mathbf{P}_{k|k-1} \quad (70)$$

8 Odometry from Encoders

8.1 Encoder Measurements

Each motor has an encoder providing:

- Drive encoders: Angular position or velocity of each wheel
- Steering encoders: Current steering angle

Encoder Count to Distance Conversion: If an encoder has *CPR* counts per revolution (after any gear reduction), the distance traveled is:

$$\Delta s = \frac{\Delta counts}{CPR} \cdot 2\pi r_{wheel} \quad (71)$$

where $r_{wheel} = 0.0127$ m is the wheel radius.

Example: For an encoder with 1000 CPR, if $\Delta counts = 500$:

$$\Delta s = \frac{500}{1000} \cdot 2\pi \cdot 0.0127 = 0.0398 \text{ m} = 39.8 \text{ mm} \quad (72)$$

8.2 Dead Reckoning

At each time step:

1. Read encoder distances: $\Delta s_{FL}, \Delta s_{FR}, \Delta s_{RL}, \Delta s_{RR}$
2. Read steering angles: $\theta_{s,measured}$
3. Compute robot displacement in robot frame
4. Transform to world frame using current heading
5. Update position

8.3 Odometry Equations

Average displacement:

$$\Delta s_{avg} = \frac{\Delta s_{FL} + \Delta s_{FR} + \Delta s_{RL} + \Delta s_{RR}}{4} \quad (73)$$

Robot frame displacement:

$$\Delta x_R = \Delta s_{avg} \cos(\theta_s) \quad (74)$$

$$\Delta y_R = \Delta s_{avg} \sin(\theta_s) \quad (75)$$

Rotation from differential drive (if applicable):

$$\Delta \psi_{enc} = \frac{(\Delta s_{FR} + \Delta s_{RR}) - (\Delta s_{FL} + \Delta s_{RL})}{w} \quad (76)$$

World frame update:

$$x_{k+1} = x_k + \Delta x_R \cos \psi - \Delta y_R \sin \psi \quad (77)$$

$$y_{k+1} = y_k + \Delta x_R \sin \psi + \Delta y_R \cos \psi \quad (78)$$

$$\psi_{k+1} = \psi_k + \Delta \psi \quad (79)$$

9 Control Implementation

9.1 System Architecture

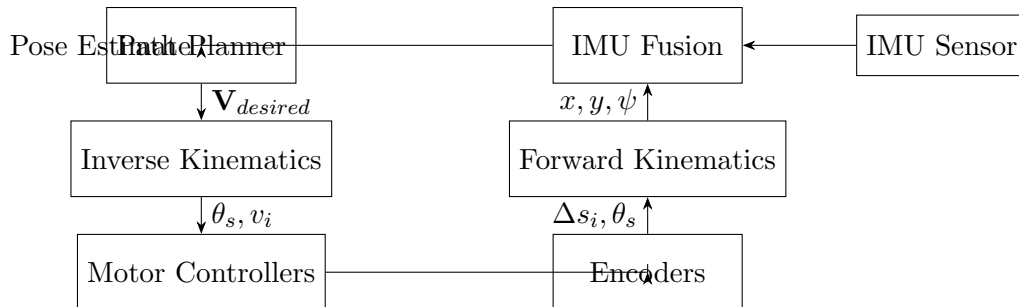


Figure 5: Control system architecture showing the flow from path planning to motor control and feedback through odometry and IMU fusion.

9.2 Control Loop Structure

Main Loop (100 Hz):

```

void loop() {
    // 1. Read encoders
    readEncoders(&delta_s[4], &theta_s);

    // 2. Update odometry
    updateOdometry(delta_s, theta_s);

    // 3. IMU update (every 5 cycles = 20 Hz)
    if (loop_count % 5 == 0) {
        updateIMU();
    }
}
  
```

```

        fuseHeading();
    }

    // 4. Path planning / high-level control
    computeDesiredVelocity(&v_x, &v_y, &omega);

    // 5. Inverse kinematics
    inverseKinematics(v_x, v_y, omega,
                     &theta_s_cmd, wheel_speeds);

    // 6. Motor control
    setSteeringAngle(theta_s_cmd);
    setWheelSpeeds(wheel_speeds);

    loop_count++;
    delay_microseconds(10000); // 10ms = 100Hz
}

```

9.3 Arduino Nano 33 BLE Feasibility

Specifications:

- Processor: nRF52840 (ARM Cortex-M4, 64 MHz)
- RAM: 256 KB
- Flash: 1 MB
- Built-in 9-DOF IMU (LSM9DS1)

Computational Load:

- Encoder reading: $\sim 10 \mu s$ per encoder
- Odometry update: $\sim 50 \mu s$
- IMU reading + fusion: $\sim 500 \mu s$ (20 Hz)
- Inverse kinematics: $\sim 100 \mu s$
- Motor control (PWM): $\sim 20 \mu s$

Total per cycle: $\sim 700 \mu s \ll 10,000 \mu s$ (100 Hz period)

Conclusion: **Feasible** with plenty of headroom for additional processing.

9.4 Motor Speed Requirements

Given the wheel radius $r_{wheel} = 0.0127$ m (12.7 mm), we can calculate required motor speeds for typical operations:

Operation	Linear Speed	Angular Speed	RPM
Slow (tight gaps)	0.02 m/s	1.57 rad/s	15.0 RPM
Medium (corridors)	0.05 m/s	3.94 rad/s	37.6 RPM
Fast (open areas)	0.10 m/s	7.87 rad/s	75.2 RPM
Maximum safe	0.15 m/s	11.81 rad/s	112.8 RPM

Table 2: Required motor speeds for different linear velocities

Motor Selection Criteria:

- Minimum speed range: 0-150 RPM (with gear reduction if needed)
- Torque requirement: Depends on robot weight and acceleration
- Encoder resolution: At least 100 CPR for adequate position feedback
- Recommended: DC gear motors with 100-300 RPM no-load speed

10 Maze Navigation Strategy**10.1 Navigation Phases****10.1.1 Phase 1: Straight Corridor**

- Mode: Crab ($\theta_s = 0$)
- Speed: 0.05 - 0.1 m/s
- Control: Simple distance-based, correct heading with IMU

10.1.2 Phase 2: 90° Turn

- Mode: Pivot ($\theta_s = 45$)
- Angular velocity: 0.1 - 0.2 rad/s
- Control: IMU-based angle control
- Target: $\psi_{target} = \psi_{current} + 90$

10.1.3 Phase 3: Centering in Corridor

- Use lateral sensors (ultrasonic/IR)
- Mode: Crab at small angles ($\theta_s = \pm 10 - 20$)
- Correct position while moving forward

10.2 Tight Gap Navigation

For 180mm gaps with 119mm robot:

Clearance budget:

$$\text{Total clearance} = 180 - 119 = 61 \text{ mm} \quad (\pm 30.5 \text{ mm per side}) \quad (80)$$

Required accuracy:

- Position: ± 15 mm
- Heading: ± 2 (at 0.3 m depth, $\Delta x = 0.3 \tan(2) \approx 10$ mm)

Strategy:

1. Approach gap with precise centering
2. Use IMU to maintain heading
3. Slow speed (0.02-0.03 m/s) for control
4. Continuous lateral correction using sensors

11 Implementation Checklist

11.1 Hardware Setup

- ☐ Mount Arduino Nano 33 BLE Sense Rev2
- ☐ Connect 4 drive motor controllers
- ☐ Connect 2 steering motor controllers (left/right)
- ☐ Wire all encoders (4 drive + 2 steering)
- ☐ Calibrate IMU (gyro zero-rate, magnetometer hard/soft iron)
- ☐ Add lateral distance sensors (optional but recommended)

11.2 Software Development

- ☐ Implement encoder reading with interrupts
- ☐ Write inverse kinematics functions (3 modes)
- ☐ Implement forward kinematics (odometry)
- ☐ Add IMU reading and complementary filter
- ☐ Write motor control (PWM + direction)
- ☐ Implement 100 Hz main control loop
- ☐ Add path planning / state machine

11.3 Testing Phases

- ☐ Test individual motor control
- ☐ Calibrate steering angle sensors
- ☐ Test pivot mode (measure actual rotation)
- ☐ Test crab mode (measure straight-line accuracy)
- ☐ Test IMU fusion (compare heading over time)
- ☐ Test odometry accuracy (known distance)
- ☐ Test full maze navigation

12 Summary

12.1 Key Equations

Mode	Equations
Pivot	$\theta_s = 45, v_i = \omega r \cdot \text{sign}_i, r = 0.0842 \text{ m}$
Crab	$\theta_s = \arctan 2(v_y, v_x), \text{ all } v_i = v$
Ackermann	$\theta_s = \arctan(L\omega/v), v_{inner/outer} = \omega(R \mp w/2)$
IMU Fusion	$\psi = 0.98(\psi + \omega_{gyro}\Delta t) + 0.02\psi_{mag}$
Odometry	$\Delta x = (\Delta s \cos \theta_s) \cos \psi - (\Delta s \sin \theta_s) \sin \psi$

Table 3: Summary of key kinematic equations for each mode

12.2 Recommended Parameter Values

- Wheel radius: $r_{wheel} = 0.0127 \text{ m}$ (12.7 mm)
- Pivot mode angular velocity: 0.1 - 0.2 rad/s (6-12 RPM at wheels)
- Crab mode linear velocity: 0.05 - 0.1 m/s (38-75 RPM) / 0.02 - 0.03 m/s (15-23 RPM in tight gaps)
- Control loop frequency: 100 Hz
- IMU update frequency: 20 Hz
- Complementary filter alpha: 0.98
- Heading tolerance: ± 2
- Position tolerance: $\pm 15 \text{ mm}$

13 Conclusion

This document provides a complete kinematic model for your 4-wheel synchronized steering maze robot. The mechanical constraint $\theta_l = -\theta_r$ simplifies control while enabling three useful modes:

1. **Pivot:** For 90° turns in tight spaces
2. **Crab:** For straight-line motion and precise positioning
3. **Ackermann:** For smooth curves when space permits

The combination of encoder-based odometry and IMU fusion provides robust position estimation for precise navigation through 180mm gaps. The Arduino Nano 33 BLE Sense Rev2 has sufficient computational power for 100 Hz control with IMU updates at 20 Hz.

Next Steps:

1. Implement basic motor control and encoder reading
2. Add pivot mode with IMU-based angle control
3. Implement crab mode with distance-based stopping

4. Integrate sensor fusion for heading correction
5. Add path planning for maze navigation

Good luck with your maze navigation robot!